Partie 3 : Intégrer Keycloak avec Spring Security en utilisant JWT

🧩 Objectif :

Laisser **Keycloak gérer l'authentification** (login, logout, tokens, rôles...)
Et que ton **backend Spring valide et autorise** les requêtes via **JWT signé**.

✅ Étapes pour intégrer Keycloak + Spring Security (JWT)

Étape 1 – Côté Keycloak

1. **Créer un Realm**
   Exemple : `inventora`
2. **Créer un Client**
   - `Client ID` : `spring-boot-backend`
   - Type : `Confidential` ou `Public` selon besoin
   - Activer : `Standard Flow` (pour login)
   - **Cocher "Access Type: bearer-only"** si backend sans UI
3. **Créer des utilisateurs** avec des rôles :
   - Exemple : `admin`, `user`
   - Attribuer les rôles dans la section **"Role Mappings"**
4. **Configurer les rôles dans le token**
   - Aller dans le Client → Mappers
   - Créer un mapper :
     - **Name**: `realm roles`
     - **Mapper Type**: `User Realm Role`
     - **Token Claim Name**: `realm_access.roles`
     - Cocher : Add to ID token, Add to access token, Add to userinfo

---

Étape 2 – Côté Spring Boot

🔧 **Ajouter les dépendances**

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

```yaml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:8080/realms/inventora
```

> 🔐 Spring va automatiquement aller chercher la **clé publique (JWKS)** exposée par Keycloak pour **vérifier les tokens**.

Étape 3 – Configurer SecurityFilterChain

```java
@Configuration
@EnableMethodSecurity
public class SecurityConfig {

  @Bean
  public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
      .authorizeHttpRequests(auth -> auth
        .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
        .requestMatchers(HttpMethod.GET, "/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated()
      )
      //.oauth2ResourceServer(oauth2 -> oauth2.jwt())
      .oauth2ResourceServer(oauth2 -> oauth2
            .jwt(jwt -> jwt.jwtAuthenticationConverter(jwtAuthenticationConverter())))
      .csrf(csrf -> csrf.disable());

    return http.build();
  }
}

```

Étape 4 – Ajouter le converter pour les rôles

Spring attend des rôles en `ROLE_ADMIN`, mais Keycloak les envoie comme `"admin"`. Donc :

```java
@Bean
public JwtAuthenticationConverter jwtAuthenticationConverter() {
  JwtGrantedAuthoritiesConverter rolesConverter = new JwtGrantedAuthoritiesConverter();
  rolesConverter.setAuthorityPrefix("ROLE_");
  rolesConverter.setAuthoritiesClaimName("realm_access.roles");

  JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
  converter.setJwtGrantedAuthoritiesConverter(rolesConverter);
  return converter;
}
```

Étape 5 – Sécuriser tes endpoints

```java
@RestController
public class DemoController {

  @GetMapping("/admin/hello")
  @PreAuthorize("hasRole('ADMIN')")
  public String adminHello() {
    return "Hello Admin!";
  }

  @GetMapping("/user/hello")
  public String userHello() {
    return "Hello User!";
  }
}

```

✅ Résultat

- 🔐 Keycloak gère : login, token JWT, utilisateurs, rôles
- ✅ Spring valide automatiquement le JWT reçu dans le header
- ✅ Pas besoin d'écrire de filtre ou de service d'auth

Élément	Spring Security "classique"	Avec Keycloak
Login	Formulaire Spring, ou `/login`	Géré par Keycloak (OpenID Connect)
AuthManager	Utilisé pour valider login/pass	❌ **Pas utilisé**
UserDetailsService	Charge user depuis la base	❌ **Pas utilisé**
Token	UUID ou JWT généré côté Spring	✅ **JWT signé généré par Keycloak**
Rôles	Récupérés via `UserDetails`	✅ Récupérés depuis le token JWT (Keycloak)
AuthProvider	Compare login/pass	❌ Plus nécessaire (Keycloak le fait)

```
[Utilisateur se connecte via Keycloak UI]
                    ⬇
     🔐 Keycloak valide les identifiants
                    ⬇
   ✅ Keycloak génère un JWT signé (avec roles, infos user...)
                    ⬇
[Le frontend ou Postman envoie ce token à l'API Spring]
                    ⬇
 📦 Spring reçoit : Authorization: Bearer <token>
                    ⬇
    Spring Security vérifie le token automatiquement
        (signature, date, contenu, rôles...)
                    ⬇
 🔐 JWT converti en Authentication via JwtAuthenticationConverter
                    ⬇
     Authentication mis dans le SecurityContext
                    ⬇
        ✅ Accès accordé ou refusé selon les rôles
```

🧠 Schéma de l'authentification dans Spring Security (simplifié)

```
             [ Requête HTTP avec login + password ]
                              ⬇
                      🔒 AuthenticationManager
                              ⬇
               🔁 appelle un ou plusieurs AuthenticationProvider
                              ⬇
     🔍 AuthenticationProvider utilise UserDetailsService
                              ⬇
              🔎 Charge UserDetails depuis la base ou mémoire
                              ⬇
         🔐 Vérifie le mot de passe (via PasswordEncoder)
                              ⬇
            ✅ Retourne un objet Authentication (rempli)
```

```
                              ⬇
            📦 Mis dans le SecurityContextHolder
                              ⬇
         🎯 Accès autorisé ou refusé selon les rôles
```